7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **4**
1.1 Obiettivo del documento 4
1.2 Glossario 4
1.3 Riferimenti 4
1.3.1 Riferimenti normativi 4
1.3.2 Riferimenti informativi 4

2 Tecnologie utilizzate **6**

3 Contesto del sistema **7**
3.1 Modello architetturale 7
3.2 Descrizione degli attori esterni 7
3.3 Descrizione dei sistemi esterni 7

4 Architettura dei container **8**
4.1 [Container 1] 8
4.1.1 Tecnologia 8
4.2 [Container 2] 8
4.2.1 Tecnologia 8
4.3 [Container 3] 8
4.3.1 Configurazione 8

5 Architettura delle componenti **9**
5.1 [Componente 1] 9
5.1.1 Descrizione 9
5.1.2 Interfacce 9
5.1.3 Dipendenze 9
5.1.4 Tracciamento dei requisiti 9
5.2 [Componente 2] 9
5.2.1 Descrizione 9
5.2.2 Interfacce 9
5.2.3 Dipendenze 9
5.2.4 Tracciamento dei requisiti 9

6 Componente Frontend: Modulo Import **10**
6.1 Descrizione 10
6.1.1 Lista delle Classi e Strutture Dati 10
6.2 Interfacce 12
6.2.1 IDipFacade 12
6.2.2 IDipChannel 13
6.2.3 DipIpcGateway 13
6.3 Dipendenze 14
6.4 Tracciamento dei requisiti 14

Elenco delle figure

Elenco delle tabelle

2 Tracciamento Requisiti Funzionali - Modulo Import 14

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

3 Contesto del sistema

3.1 Modello architetturale

3.2 Descrizione degli attori esterni

3.3 Descrizione dei sistemi esterni

4 Architettura dei container

4.1 [Container 1]

4.1.1 Tecnologia

Descrizione

4.2 [Container 2]

4.2.1 Tecnologia

Descrizione

4.3 [Container 3]

4.3.1 Configurazione

5 Architettura delle componenti

5.1 [Componente 1]

5.1.1 Descrizione

5.1.2 Interfacce

5.1.3 Dipendenze

5.1.4 Tracciamento dei requisiti

5.2 [Componente 2]

5.2.1 Descrizione

5.2.2 Interfacce

5.2.3 Dipendenze

5.2.4 Tracciamento dei requisiti

6 Componente Frontend: Modulo Import

6.1 Descrizione

Il modulo Import gestisce l'*application logic* dedicata all'esplorazione gerarchica dei pacchetti di archiviazione. Il componente permette la navigazione (*lazy loading*) delle classi documentali, dei processi e dei documenti, fornendo inoltre una funzionalità di anteprima (*preview*) dei contenuti e dei relativi metadati. L'architettura è basata sul pattern Smart-Dumb Components per la UI e sul pattern Facade per il coordinamento dei servizi.

6.1.1 Lista delle Classi e Strutture Dati

ImportState Oggetto responsabile della gestione dello stato reattivo del modulo tramite Angular Signals. Implementa la *Persistence Logic* del frontend mantenendo uno snapshot sincronizzato dell'applicazione.

Descrizione degli attributi della struttura:

- **phase Signal<ImportPhase>**: segnale che rappresenta la fase attuale del modulo (IDLE, LOADING, READY, EMPTY, ERROR);
- **rootNodes Signal<DipTreeNode[]>**: contiene la lista dei nodi radice (classi documentali) del DIP caricato;
- **nodeCache Signal<Map<string, DipTreeNode[]»**: cache reattiva che memorizza i figli dei nodi già esplorati per ottimizzare le prestazioni ed evitare chiamate IPC ridondanti;
- **nodeChildrenErrors Signal<Map<string, AppError»**: mappa che associa a ogni nodo l'eventuale errore riscontrato durante il caricamento dei suoi figli;
- **selectedDocument Signal<DipTreeNode | null>**: memorizza il riferimento al documento attualmente selezionato dall'utente per la visualizzazione;
- **indicizzazione Signal<StatoIndicizzazione>**: traccia lo stato dell'indicizzazione semantica del DIP (es. NON_COMPLETATA, IN_CORSO, COMPLETATA);
- **loading Signal<boolean>**: indica se è in corso un'operazione asincrona di recupero dati.

Descrizione dei metodi invocabili dalla struttura:

- **setRootNodes(nodes: DipTreeNode[])**: aggiorna i nodi radice e imposta la fase su READY o EMPTY in base al contenuto;

- **setChildrenForNode(nodeId: string, children: DipTreeNode[])**: popola la cache dei nodi figli per un determinato nodo padre;
- **setChildrenError(nodeId: string, error: AppError)**: registra un errore specifico per il caricamento dei figli di un nodo;
- **setSelectedDocument(node: DipTreeNode | null)**: aggiorna il documento visualizzato nella preview;
- **setIndicizzazione(stato: StatoIndicizzazione)**: aggiorna lo stato del processo di indicizzazione;
- **reset()**: riporta lo stato del modulo alle condizioni iniziali, pulendo cache e nodi.

ImportFacade Rappresenta l'oggetto che implementa la *business logic* di coordinamento del modulo Import. Agisce come mediatore tra i componenti dell'interfaccia utente, lo stato reattivo e il layer di infrastruttura (Gateway). Implementa l'interfaccia *IDipFacade*.

Descrizione degli attributi della struttura:

- **importState ImportState**: istanza del servizio di gestione dello stato reattivo del modulo;
- **ipcGateway DipIpcGateway**: istanza del gateway per la comunicazione IPC con il processo Main di Electron.

Descrizione dei metodi invocabili dalla struttura:

- **loadRootNodes() Promise<void>**: coordina il caricamento iniziale delle classi documentali radice del DIP. Gestisce l'impostazione dello stato di caricamento, l'invocazione del gateway, il mapping dei DTO in modelli di dominio tramite *DipMapper* e la gestione centralizzata degli errori;
- **loadChildren(nodeId string) Promise<void>**: implementa la logica di *lazy loading* per i nodi figli di un elemento dell'albero. Il metodo verifica preventivamente la presenza dei dati nella cache dello stato per ottimizzare le prestazioni; in caso di assenza, interroga il gateway e aggiorna la cache o registra l'errore specifico per il nodo;
- **selectDocument(node DipTreeNode) void**: permette di impostare il documento attualmente selezionato dall'utente all'interno dello stato, scatenando l'aggiornamento reattivo dei componenti di anteprima;
- **retryLoad() void**: esegue il ripristino del modulo (*reset*) e tenta nuovamente il caricamento delle classi radice a seguito di un errore critico;
- **clearCache() void**: esegue la pulizia completa della cache dei nodi e degli errori memorizzati, riportando lo stato alle condizioni iniziali.

Descrizione dei segnali esposti:

- **rootNodes**: fornisce l'accesso in sola lettura alla lista dei nodi radice;
- **selectedDocument**: fornisce il riferimento al documento correntemente selezionato;
- **loading**: indica se è in corso un'operazione asincrona di recupero dati;
- **phase**: espone la fase attuale del ciclo di vita del modulo (es. READY, ERROR, LOADING).

DipTreeNode (Domain Model) Rappresenta l'oggetto di dominio per la resa visiva dell'albero.

- **id string**: identificativo univoco;
- **type NodeType**: enumerazione (CLASSE, PROCESSO, DOCUMENTO);
- **label string**: nome visualizzato del nodo.

6.2 Interfacce

6.2.1 IDipFacade

Rappresenta l'interfaccia di contratto della Business Logic per il modulo Import. Definisce le proprietà reattive (*Signals*) per l'osservazione dello stato e i metodi necessari per l'interazione con i dati del DIP.

Descrizione delle proprietà dell'interfaccia:

- **rootNodes Signal<DipTreeNode[]>**: segnale per l'accesso alla lista dei nodi radice caricati;
- **selectedDocument Signal<DipTreeNode | null>**: segnale che espone il documento attualmente selezionato per l'anteprima;
- **loading Signal<boolean>**: segnale booleano che indica lo stato di avanzamento delle operazioni asincrone;
- **phase Signal<ImportPhase>**: segnale che espone la fase corrente del ciclo di vita del modulo.

Descrizione dei metodi dell'interfaccia:

- **loadRootNodes() Promise<void>**: definisce l'operazione di caricamento delle classi documentali di primo livello;
- **loadChildren(nodeId string) Promise<void>**: definisce l'operazione di recupero dei nodi figli (processi o documenti) associati a un identificativo;
- **selectDocument(node DipTreeNode) void**: definisce la logica di selezione di un nodo per la visualizzazione dei dettagli;
- **retryLoad() void**: definisce la procedura di ripristino e nuovo tentativo di caricamento del modulo;
- **clearCache() void**: definisce l'operazione di pulizia dello stato e della cache locale.

6.2.2 IDipChannel

Interfaccia di contratto per il layer di infrastruttura. Definisce le operazioni di basso livello necessarie per l'interrogazione del sistema operativo tramite i canali IPC di Electron.

Descrizione dei metodi dell'interfaccia:

- **getClasses() Promise<ClasseDocumentaleDto[]>**: definisce la richiesta per ottenere le classi documentali radice del DIP;
- **loadChildren(nodeId string) Promise<DipTreeNodeDto[]>**: definisce la richiesta per ottenere i nodi figli dato un identificativo;
- **downloadFile(nodeId string) Promise<Blob>**: definisce la richiesta di estrazione e salvataggio di un file fisico.

6.2.3 DipIpcGateway

Rappresenta l'Adapter di infrastruttura incaricato di gestire la comunicazione asincrona verso i canali IPC di Electron. Accede all'oggetto bridge esposto dal *preload script* tramite `globalThis.electron`.

Descrizione dei metodi invocabili dalla struttura:

- **getClasses()**: esegue l'invocazione del canale `browse:get-document-class-by-dip-i` per recuperare le classi radice;
- **loadChildren(nodeId string)**: esegue il parsing dell'identificativo testuale in numerico e invoca il canale `browse:get-documents-by-process`;
- **downloadFile(nodeId string)**: coordina la richiesta di download invocando il canale `file:download` con un percorso di destinazione predefinito.

Descrizione dei metodi statici della struttura:

- **classeToDipTreeNode(dto DipClassDto) DipTreeNode**: mappa un DTO di tipo classe documentale nel modello di dominio, impostando correttamente la tipologia del nodo;
- **toDipTreeNode(dto DipNodeDto) DipTreeNode**: metodo generico che trasforma i dati di un processo o di un documento nel corrispondente oggetto *DipTreeNode*.

6.3 Dipendenze

Il modulo presenta le seguenti dipendenze principali:

- **Angular Signals**: utilizzato per la gestione della reattività dello stato e la sincronizzazione della UI;
- **Electron IPC Bridge**: interfaccia di comunicazione nativa utilizzata dal *DipIpcGateway* per interagire con il sistema operativo;
- **AppError**: modello condiviso per la gestione e la propagazione degli errori di sistema e di comunicazione.

6.4 Tracciamento dei requisiti

La tabella seguente mappa i requisiti funzionali obbligatori relativi all'Import sulle componenti realizzate.

Codice	Descrizione Sintetica	Componente Software
R-1-F-Ob	Visualizzazione elenco classi documentali.	*ImportFacade, DipTree*
R-2-F-Ob	Messaggio di errore per DIP vuoto.	*ImportState, DipLoadingPage*
R-7-F-Ob	Visualizzazione processi di una classe.	*ImportFacade, DipTreeNode*
R-11-F-Ob	Visualizzazione documenti di un processo.	*ImportFacade, DipTreeNode*
R-21-F-Ob	Anteprima del documento selezionato.	*DocumentPreview, ImportFacade*
R-26-F-Op	Visualizzazione stato indicizzazione semantica.	*ImportState, ImportFacade*

Tabella 2: Tracciamento Requisiti Funzionali - Modulo Import